

FROM ROUTING EFFICIENCY TO ADAPTIVE INSTITUTIONAL PRIORITIES

THE SELF-DRIVING DELIVERY NETWORK: OPTIMIZATION WITH JUDGMENT

A Synthetic Logistics Simulator and Its Transfer to Finance, Government and Complex Corporations

A pedagogical study of how deterministic optimization, environmental change, bounded LLM judgment, governance, execution and feedback combine when priorities can override an original sequence—and why the same architecture matters for capital allocation, public programs, liquidity, risk and multi-subsidiary institutions.

CENTRAL PROPOSITION

Optimization determines an efficient sequence under a specified objective and state of the world. Judgment becomes necessary when the state changes and the institution must decide which objective now deserves priority. The delivery network makes this distinction observable: disruptions do not merely require a new route; they can change the ordering of commitments. This provides a transferable architecture for finance, government and complex corporations in which priorities, constraints and timing interact dynamically.

Alejandro Reynoso

AI for Financial Practitioners / Autonomous Systems / Institutional Design

October 2026

Pedagogical Research Monograph

Educational simulator. The network, packages, service classes, disruptions, costs and decisions are synthetic teaching constructs. The monograph studies architecture and transferability, not industrial parcel-routing performance.

Abstract

This monograph studies a synthetic self-driving parcel-delivery network as a laboratory for a broader class of autonomous institutions. The notebook begins with a deterministic operating plan: a fleet, heterogeneous packages, deadlines, capacity constraints and an initial dispatch sequence. It then introduces congestion, weather, vehicle failures, urgent arrivals and customer changes. These shocks create a problem that is deeper than re-optimization. The institution must determine whether the original ordering of objectives still deserves to govern action.

The architecture deliberately separates quantitative optimization from bounded judgment. Deterministic code estimates travel time, slack, capacity and risk; constructs feasible alternatives; and preserves hard constraints. Claude Haiku 4.5 is introduced as a supervisory judgment layer that selects among already-feasible alternatives according to an institutional hierarchy. A governance gate independently validates the recommendation before execution. Feedback then reconstructs the operating state and closes the loop. Cell 16 adds an ex-post counterfactual layer: it asks whether LLM judgment actually changed decisions relative to a deterministic baseline and requests an evidence-bounded narrative explaining the differences.

The central transfer insight is that many financial and public-sector systems are scheduling and allocation systems under changing priorities. A federal government allocates budget authority, administrative attention, procurement capacity and implementation time across programs that do not share a common calendar. A complex corporation allocates capital, liquidity, management attention and risk capacity across subsidiaries. A bank allocates balance-sheet capacity, collateral and liquidity across businesses. In each case, an original optimized sequence may become inappropriate after a contingency. The relevant autonomous architecture is therefore not “LLM replaces optimizer,” but *optimization with judgment*: algorithms preserve feasibility and measurement; judgment reprioritizes among legitimate objectives; governance constrains authority; and feedback continuously revises the state.

RESEARCH QUESTION

How should an autonomous institution behave when a plan that was optimal yesterday remains computationally feasible today, but environmental change makes its original priority ordering institutionally inappropriate?

COMPUTATIONAL COMPANION — COLAB NOTEBOOK

The complete executable companion to this monograph is `Self_Driving_Delivery_Network_LLM.ipynb`. It contains the synthetic delivery network, disruption engine, deterministic optimization baseline, bounded Claude Haiku 4.5 judgment gateway, governance and execution loop, feedback mechanism, and the Cell 16 ex-post evaluation of whether judgment changed operational choices. In the Prism/OpenAI project, place the notebook in the Files section alongside this LaTeX source. When the PDF is compiled locally with the notebook in the same directory, the filename below acts as the computational reference:

`Self_Driving_Delivery_Network_LLM.ipynb`

Contents

1	1. Why Delivery Is a Model of Institutional Autonomy	5
1.1	The architectural ladder	5
1.2	Why the distinction matters	5
2	2. Cells 1–6: Build the World, Optimize, Then Break the Assumptions	7
2.1	Cells 1–3: environment, data and baseline plan	7
2.2	Cells 4–6: heterogeneity, risk and initial dispatch	7
2.3	Why a transparent baseline matters	7
3	3. Cells 7–12: From Exceptions to Bounded Judgment	9
3.1	Disruptions, perception and diagnostics	9
3.2	Candidate generation and the deterministic baseline	9
3.3	Cell 12: Claude as a bounded supervisory judge	9
3.4	Keeping judgment inside a verifiable boundary	9
4	4. Cells 13–16: Governance, Closed-Loop Control and the Test of Judgment	11
4.1	Governance before execution	11
4.2	Feedback and the integrated experiment	11
4.3	Cell 16: did judgment actually matter?	11
4.4	Closing the loop without hiding responsibility	11
5	5. Optimization with Judgment: A General Model	13
5.1	A layered decision model	13
5.2	Operational tests for the architecture	14
6	6. Transfer I: The Federal Government as a Dynamic Allocation System	15
6.1	Public allocation as a constrained sequence	15
7	7. Transfer II: Complex Corporations and Multi-Subsidiary Groups	17
7.1	Group-level coordination and entity boundaries	17
8	8. Transfer III: Finance—Where Re-Sequencing Is Already Everywhere	19
8.1	Portfolio management and trading	19
8.2	Bank balance-sheet and liquidity management	19
8.3	Collateral and margin management	19
8.4	Credit and restructuring workflows	19
8.5	Insurance and claims	19
8.6	Treasury, settlement and payment operations	19
8.7	Finance-specific implementation conditions	19
9	9. A Taxonomy of Transfer Learning from the Delivery Network	21
9.1	What transfers, and what must be rebuilt	21
10	10. Governance: Why Judgment Must Not Become Unbounded Discretion	23
10.1	A control stack for bounded judgment	23
11	11. What Cell 16 Adds: From Demonstration to Research Design	25

11.1 A stronger evaluation protocol	25
12 12. Conclusion	27
12.1 Implications for institutional design	27
13 References	29

1. Why Delivery Is a Model of Institutional Autonomy

A delivery network looks at first like a classical operations-research problem. Vehicles must be assigned to packages; routes must respect capacity; deadlines must be met; distance and cost should be minimized. The notebook preserves that classical foundation. Yet its pedagogical purpose begins where the static optimization problem ends.

The operating environment changes after the initial plan is produced. Congestion alters travel times. Weather changes the effective network. A vehicle failure removes capacity. A customer changes a delivery window. An urgent medical package arrives after the day's routes have already been formed. These events matter because commitments are heterogeneous. A medical shipment, a priority package and an ordinary delivery are not simply three observations with different coefficients; they can embody different institutional priorities.

OPTIMIZATION VERSUS JUDGMENT

An optimizer answers: *given the objective function and constraints, which feasible action scores best?* Judgment answers a different question: *given several legitimate objectives and a changed environment, which objective should dominate now?* The first problem is computational. The second is institutional.

This distinction echoes bounded rationality: organizations do not operate with a single timeless objective and unlimited foresight; they work through rules, routines, attention and adaptation [11; 7]. It also connects with dynamic decision-making under uncertainty, where policies must be revised as new information arrives [5; 6]. The novelty of the notebook is to make the separation operational and auditable inside one closed loop.

Illustration summary: Cells 1–3 establish the computational environment, synthetic network and initial optimization. This is the “day zero” plan: efficient under the state known at initialization.

The architectural ladder

The notebook can be read as a progression through six forms of institutional intelligence:

1. **Representation:** packages, vehicles, geography and commitments become explicit state.
2. **Optimization:** a baseline plan sequences resources against deadlines and capacity.
3. **Perception and analysis:** disruptions are converted into revised ETAs, slack and risk.
4. **Judgment:** competing feasible actions are evaluated under a hierarchy of priorities.
5. **Governance and execution:** recommendations are validated before they can change the world.
6. **Feedback and evaluation:** outcomes update the state and the role of judgment is measured ex post.

Why the distinction matters

The distinction between a changed route and a changed priority is important because the two call for different evidence and different authority. Congestion may make one route longer, but it does not by itself establish that a medical delivery should outrank every other commitment. A new urgent request may change the service hierarchy, but only if its identity, deadline and eligibility are verified. The operating system therefore needs a disciplined chain: observe the event, estimate its operational consequences, determine whether it activates a pre-authorized priority rule, and then choose a feasible response. Treating every disruption as a reason to invoke a language model would confuse novelty with importance and could make routine operations slower without improving decisions.

The synthetic setting makes that chain visible by keeping the entities small and the events legible. A vehicle breakdown changes available capacity; a road delay changes estimated arrival time; an urgent package may change the relative cost of delay. Each event enters the state through a different field, and each can be checked against the original commitment. In a larger institution, the same discipline means distinguishing observed facts from forecasts and policy choices. A liquidity report is not itself a decision to stop lending. A storm forecast is not itself authority to move funds. A risk signal should prompt a defined diagnostic and review, not an untraceable change in objectives.

The model is intentionally a teaching construct rather than a claim about industrial parcel routing. Its simplified geography, small candidate set and simulated shocks are useful precisely because they expose the architecture without hiding it inside a commercial solver. It cannot establish that an LLM improves real delivery performance, nor can the notebook's synthetic outcomes establish that the same priority rules are appropriate for a public agency or bank. Those are separate empirical and institutional questions. The transferable result is a design pattern: hold the resource and constraint model explicit, create a stable baseline, observe the environment, and make priority changes visible as distinct decisions.

A practical reader should consequently evaluate the system at several levels. Does it represent the relevant commitments correctly? Can it detect that an assumption has changed? Are all actions available to the judgment layer genuinely permissible? Does the recommendation identify its operative reason? Can an independent control reject it? Finally, does the recorded outcome show whether the intervention helped the intended service objective without creating unacceptable cost or inequity elsewhere? These questions make autonomy measurable. They also prevent a visually impressive recommendation from being mistaken for a reliable institution.

2. Cells 1–6: Build the World, Optimize, Then Break the Assumptions

Cells 1–3: environment, data and baseline plan

Cell 1 establishes reproducibility and the Claude connection. In the revised notebook the secret is read as `ANTHROPIC_API_KEY`, connectivity is tested explicitly, and the experiment is prevented from silently presenting a deterministic run as an LLM-driven one. Cell 2 defines packages, vehicles, disruptions, candidate actions and decisions as distinct objects. This is an important control principle: model output is not itself mutable operational state. Cell 3 creates an abstract city whose geometry is deliberately simple enough that the architecture remains visible.

The baseline plan is not intended to compete with industrial vehicle-routing solvers. Its purpose is to create a coherent sequence of commitments that can later be disturbed. This follows the long operations-research tradition in routing and scheduling while keeping the experiment transparent [13].

Cells 4–6: heterogeneity, risk and initial dispatch

The synthetic workload introduces standard, priority and medical packages, heterogeneous weights and deadlines, and finite vehicle capacity. Cell 5 computes travel time, deadline slack and a risk score deterministically. Cell 6 then creates the initial dispatch. This division of labor is intentional: arithmetic, feasibility and repetitive optimization remain in code.

Illustration summary: Cells 4–6 make the environment dynamic. Disruptions alter the current state, and exception detection identifies cases in which the original plan may no longer reflect institutional priorities.

A useful way to formalize the transition is to distinguish a baseline objective

$$\min_{a \in \mathcal{A}(s_t)} C(a; s_t),$$

from a context-dependent priority ordering. Here s_t is the perceived state and $\mathcal{A}(s_t)$ is the feasible action set. In a static system, C may encode travel time, cost and lateness. In an adaptive institution, however, the effective objective can itself depend on the contingency z_t :

$$\min_{a \in \mathcal{A}(s_t)} \sum_{k=1}^K w_k(s_t, z_t) C_k(a; s_t).$$

The hard problem is no longer only finding a . It is determining when the weights w_k should change, which priorities are lexicographic rather than compensatory, and which changes require human authority.

Why a transparent baseline matters

The initial six cells establish more than a collection of data structures. They define what the experiment means by a commitment, a resource, a constraint and an outcome. If package classes are ambiguous, later claims about protected service cannot be tested. If vehicle capacity is recorded inconsistently, a candidate may appear feasible only because the state representation omitted a load. If deadlines are not explicit, lateness metrics cannot reveal which promises were broken. The baseline is therefore a compact contract among the simulator's components: each stage receives a known set of fields, applies a documented transformation, and returns information that the next stage can inspect.

Reproducibility is part of this contract. A fixed random seed allows the same network and shocks to be regenerated, so a difference between policies can be traced to the policy rather than to a different draw of traffic or demand. Reproducibility does not make a scenario representative of real operations, but it does make comparisons interpretable. A useful experiment should preserve the seed, initial state, event sequence, candidate set and software configuration with each run. If any of those change, the record should say so. This is especially important when a narrative explanation is produced after the result, because otherwise the explanation may describe a different case from the one the decision actually saw.

The deterministic dispatch is a benchmark, not a neutral truth. Every baseline embeds choices about the relative value of travel time, fuel, missed deadlines and reserve capacity. Those choices should be documented rather than treated as natural constants. A low-cost schedule may be more brittle because it leaves little slack; a schedule that protects slack may appear inefficient until an incident occurs. The experiment should therefore report a small set of interpretable measures side by side, such as completed commitments, lateness by service class, distance, unused capacity and disruption recovery time. A single aggregate score can conceal a policy that improves average performance by sacrificing a protected group.

There is also a pedagogical benefit to keeping the optimization modest. A sophisticated solver could make the notebook appear more realistic while making it harder to see whether the judgment layer contributes anything. Here the point is to trace why a given action was considered, which constraints ruled out alternatives and what changed after a shock. As the simulator grows, complexity should be added only when it supports a research question. More vehicles, stochastic travel times or richer depot structure may be justified; detail that does not affect the decision or its evaluation is likely to distract. This principle supports a progression from transparent toy model to validated operational model rather than a leap from demonstration to deployment.

3. Cells 7–12: From Exceptions to Bounded Judgment

Disruptions, perception and diagnostics

Cells 7–9 create the informational bridge from the world to the decision layer. The disruption generator produces reproducible shocks without revealing them to the controller before their simulated arrival. Perception applies newly observed events to mutable state. Analytical diagnostics then rank exceptions using quantitative evidence.

This separation is essential. Perception answers what appears to have changed; analysis estimates consequences; neither should decide what the institution values. In modern control language, this is analogous to maintaining a state estimate before selecting a control action [1]. In organizational terms, it separates facts and forecasts from authority.

Illustration summary: Cells 7–9 move from feasible alternatives to bounded judgment and governance. The crucial design choice is that Claude selects among alternatives rather than inventing the action space.

Candidate generation and the deterministic baseline

Cell 10 constructs a bounded set of feasible actions: retain, reassign, expedite, postpone where permitted, or escalate. Capacity and operational constraints are checked before an option reaches the judgment layer. Each alternative is annotated with service benefit, cost and fragility. Cell 11 then supplies a deterministic policy baseline.

The deterministic baseline has conceptual value even when Claude is fully available. Without a baseline, one cannot tell whether the LLM adds judgment or merely reproduces a coded ranking. The baseline therefore acts as both resilience mechanism and counterfactual benchmark.

Cell 12: Claude as a bounded supervisory judge

Cell 12 is the intellectual center of the notebook. Claude Haiku 4.5 receives a compact case and a finite set of valid candidate identifiers. It is instructed to apply an institutional hierarchy: feasibility and safety; protected service; severe deadline risk; network robustness; cost; and escalation when ambiguity is material. The output schema is intentionally small, and candidate identifiers are validated.

THE RIGHT ROLE FOR THE LLM

The LLM is not a routing engine, database, constraint solver or execution layer. Its comparative advantage is contextual prioritization among feasible actions when the objectives cannot be reduced comfortably to one stable scalar score. This is closer to supervisory discretion than to numerical optimization.

This bounded design is consistent with the broader literature on combining machine intelligence with human and institutional control, rather than treating automation as a single undifferentiated agent [2; 9]. It also reflects the practical principle that generative systems should be surrounded by explicit verification where errors can propagate into consequential actions.

Keeping judgment inside a verifiable boundary

Candidate generation is a substantive safety boundary, not a clerical preliminary. The candidate set determines which actions the language model can recommend. If it omits a safe and useful response, no quality of prose can recover that option. If it includes a physically impossible or unauthorized response, the downstream model may select it convincingly. Candidate generation should therefore be deterministic, complete enough for the stated decision, and explicit about why an option is admissible. The diagnostics should attach the relevant evidence to each option, so reviewers can compare candidates on the same basis rather than relying on the model's summary of them.

The judgment prompt should also distinguish mandatory rules from preferences. Capacity, vehicle availability and safety are hard constraints. Service protection and urgency may be lexicographic rules or policy priorities. Cost and efficiency may be trade-off criteria within the remaining feasible region. Combining all of these into one vague instruction such as “choose the best route” hides which rule controlled the result. A bounded instruction instead asks the model to identify the controlling consideration, select an existing candidate identifier, state any material uncertainty and abstain when the evidence does not support a choice. The output schema can be checked mechanically; free-form rationale is supplementary evidence, not an executable command.

This design creates testable failure modes. The model can return a malformed response, select an unknown candidate, overlook a relevant field, apply priorities inconsistently or express more confidence than the case warrants. Each should lead to a defined behavior. Invalid identifiers should be rejected; unavailable evidence should trigger an abstention or human review; service and feasibility rules should be rechecked independently before execution. A timeout or API error should not silently be recorded as a successful Claude decision. If a deterministic fallback is

authorized, the log should identify it distinctly, and evaluation should report fallback frequency separately from model performance.

Latency and operational cadence matter as well. In a simulated notebook, a delayed API response is an inconvenience. In a time-critical workflow, waiting for a language model may itself impose a cost. The system should specify which decisions can wait, which use a safe baseline after a timeout, and which must be escalated. This can be tested through fault injection: remove fields, introduce contradictory signals, cause an API failure, or provide a candidate set with no clearly dominant option. The objective is not to prove that the model never errs. It is to show that errors remain bounded, detectable and recoverable, and that the surrounding institution behaves predictably when the model is uncertain.

4. Cells 13–16: Governance, Closed-Loop Control and the Test of Judgment

Governance before execution

Cell 13 inserts a governance gate between recommendation and action. A selected candidate must still exist, remain feasible, respect protected-service rules and avoid failed vehicles or excess capacity. Execution is deterministic. Escalation returns authority to a human queue. The architecture therefore distinguishes *recommendation authority* from *execution authority*.

This is closely related to the logic of risk limits and internal controls in financial institutions: a trader, model or committee may recommend an action, but limits, mandates and independent control functions determine whether it can be implemented [3; 4].

Illustration summary: The decision core: feasible actions are generated, a deterministic baseline is preserved, and Claude supplies bounded supervisory judgment. In the current notebook, Cell 12 is the LLM judgment gateway.

Feedback and the integrated experiment

Cell 14 closes the loop. At each tick the system perceives, diagnoses, constructs candidates, requests judgment, validates, executes and records outcomes. The audit trail preserves source, confidence, rationale and result. Cell 15 makes model provenance visible: the experiment reports how many decisions were genuinely made by Claude and how many, if any, used fallback.

Illustration summary: Cells 13–15 close the autonomous loop through governance, execution, feedback and an integrated dashboard that separates operating performance from judgment provenance.

Cell 16: did judgment actually matter?

Cell 16 prevents the project from ending with a visually attractive but scientifically weak dashboard. It asks a sharper question: for the cases in which Claude acted, what would the deterministic policy have selected? The comparison is decision-point counterfactual analysis, not a full alternative-world simulation. It measures disagreement, incremental cost, service benefit and fragility, and then gives Claude a compact evidence package for an ex-post narrative.

The narrative is explicitly constrained to distinguish observed facts, counterfactual comparisons and interpretation. A different decision is not automatically a better decision. This is important: the objective is to identify where judgment changes priorities, not to declare that an LLM is superior by construction.

Illustration summary: Cell 16 evaluates the marginal role of judgment by comparing Claude's actual choices with the deterministic policy and asking for an evidence-bounded ex-post interpretation. Values shown in the infographic are illustrative unless reproduced by the current notebook run.

THE EMPIRICAL QUESTION

If Claude and the deterministic baseline always choose the same action, the LLM may be adding explanation but little decision value. If they differ selectively in high-stakes trade-offs—for example, accepting cost to protect service or reducing fragility despite a locally attractive route—then the experiment has located the margin at which judgment changes institutional behavior.

Closing the loop without hiding responsibility

A closed-loop controller is only as reliable as its account of what happened between decisions. After execution, the simulator should update vehicle location, package status, remaining capacity, elapsed time and any newly observed disruption. If it updates only the selected action but not the state consequences, the next decision is based on a fictional world. This is why feedback is not merely a dashboard feature. It is the mechanism that keeps the candidate generator and optimizer synchronized with the consequences of prior actions. The loop also gives evaluators a time-indexed record rather than a final score detached from the path that produced it.

The audit record should make the chain of responsibility reconstructable. A useful event record links the state snapshot, detected exception, candidate identifiers, deterministic baseline, model recommendation, governance result, executed action and observed outcome. It should include timestamps and provenance, while avoiding the assumption that a fluent rationale proves the decision was valid. A reviewer should be able to recalculate feasibility and compare the selected candidate with alternatives using the underlying numbers. When records are incomplete, the right conclusion is not that the model's decision was correct, but that the institution cannot establish what happened.

Cell 16 adds an important but limited counterfactual. It compares the action selected by Claude at a particular

decision point with the deterministic policy's choice under the same reconstructed state. This reveals local divergence and permits immediate comparisons of cost, service impact or fragility. It does not simulate every later consequence of following the other action. A different dispatch could change where vehicles are later located, which packages remain onboard and which future choices become available. Therefore, local counterfactual metrics should not be described as the total causal effect of the LLM policy over the full day.

A stronger analysis would replay the same event stream under both policies, allowing each policy's actions to generate its own subsequent state. Even that comparison requires careful controls: the policies must receive equivalent observations, share the same feasibility rules and be evaluated across many scenarios. For the teaching notebook, the local comparison is useful because its limitations can be explained plainly. Provenance counts, fallback rates and outcome measures should be shown together. That prevents a dashboard from implying model influence when most choices came from deterministic fallback, and it makes the governance and execution stages as visible as the recommendation itself.

5. Optimization with Judgment: A General Model

The delivery experiment suggests a general architecture for autonomous institutions. Let x_t denote the physical or financial state, y_t the perceived state, $\mathcal{A}_t$ the feasible action set, p_t the institutional priority ordering and $g(\cdot)$ the governance operator. A conventional optimizer can be represented as

$$a_t^* = \arg \min_{a \in \mathcal{A}_t} C(a \mid y_t, p_t).$$

The self-driving institutional problem adds two transitions:

$$p_{t+1} = J(y_t, p_t, z_t), \quad \tilde{a}_t = \arg \min_{a \in \mathcal{A}_t} C(a \mid y_t, p_{t+1}),$$

followed by

$$a_t = g(\tilde{a}_t, \text{rules, limits, authority}), \quad x_{t+1} = F(x_t, a_t, \varepsilon_{t+1}).$$

Here J is the judgment function: it does not create physical feasibility, but can alter the relative priority of legitimate objectives in light of context z_t . Governance g then decides whether the proposed action lies within delegated authority.

SEQUENCE IS AN OUTPUT, NOT A SACRED INPUT

Many institutions confuse the approved sequence with the objective itself. The delivery model shows why this is dangerous. The sequence was optimal conditional on an earlier state. Once conditions change, preserving the sequence mechanically may destroy the objective it was designed to serve.

The idea is related to model predictive control, in which a plan is repeatedly recomputed as the state changes [10]. The addition here is institutional: not only can forecasts and controls change; the hierarchy among objectives may also change, subject to governance.

A layered decision model

This formulation can be interpreted as a sequence of distinct questions rather than as one enlarged optimization problem. First, the measurement layer estimates the current state and identifies which observations are fresh, delayed or uncertain. Second, the feasibility layer constructs the admissible action set using physical, legal, safety and delegated-authority constraints. Third, an optimizer ranks actions against a specified objective. Fourth, a judgment process asks whether context requires a different ordering among legitimate objectives. Fifth, a governance operator confirms that the proposed action is authorized. Separating these questions makes it possible to locate disagreement: a sensor may be wrong, the feasible set may be incomplete, the objective may be stale, the priority judgment may be poor, or the approval may be invalid.

The distinction between hard constraints and priorities is particularly important. A hard constraint removes an action from the feasible set; a priority changes how remaining actions are compared. If vehicle capacity is exceeded, no amount of urgency makes that dispatch permissible. If two feasible plans both meet capacity but one protects a medical delivery, institutional policy may rank it higher even at greater distance. Treating both considerations as weighted terms in one score can allow a sufficiently large gain elsewhere to compensate for a rule that was intended to be absolute. Lexicographic ordering, explicit thresholds and policy-defined exclusions are often easier to audit than a single weighted sum.

Priority change should itself have a trigger and a scope. A weather alert might change the value of route robustness for a subset of exposed deliveries without changing the importance of every other commitment. A credit deterioration could accelerate one review without authorizing a wholesale reprioritization of the entire portfolio. The judgment function should therefore operate on a defined event, identify affected objectives and state how long the revised priority remains active. Reassessment conditions matter: a temporary emergency priority should expire or be reviewed when evidence changes, rather than becoming a permanent undocumented rule.

Uncertainty enters at several points. The state estimate may be wrong, the consequences of an action may be forecast with error, and the institution may not know whether an exception activates a policy. These uncertainties should not all be compressed into model confidence. Quantitative uncertainty can be represented through ranges, scenarios or conservative feasibility checks. Policy ambiguity may require escalation because it concerns authority rather than prediction. When evidence is weak, a system can preserve a safe option, gather more information or defer an irreversible action. The value of judgment may be to recognize that an apparently precise ranking is not robust enough to justify action.

The model also clarifies what an LLM can contribute without being given control of the entire process. It can compare structured alternatives, notice that a stated exception changes the relevance of an objective, and explain

which provided facts support its selection. It should not invent missing state, relax constraints, create candidate actions or convert a recommendation into an instruction to an actuator. These boundaries allow deterministic methods to retain tasks for which they are well suited, while judgment remains narrow, inspectable and replaceable. If later evidence shows that a stable rule handles a class of cases reliably, that class can be automated without requiring a model call on every decision.

Operational tests for the architecture

The model becomes useful only when each transition has an observable test. Measurement can be checked against independently recorded state; feasibility can be tested by constraint violations; optimization can be compared with the stated cost function; judgment can be examined for consistency across similar contingencies; governance can be audited against delegated authority; and feedback can be checked for accurate state updates. This decomposition gives reviewers a way to locate a failure instead of attributing every poor outcome to an undifferentiated “AI error.” It also supports replacement: if judgment is unreliable for a case class, the institution can route that class to a human or deterministic rule while preserving the other layers.

The architecture should make its objective hierarchy explicit enough that affected operators can challenge it. A decision record can name the triggering event, the objectives that changed rank, the candidates considered and the constraint that ruled out each rejected alternative. Such a record does not eliminate disagreement, but it clarifies whether the disagreement is about a fact, a forecast, a policy priority or an authority boundary. Those are different disputes and call for different remedies. An incorrect travel estimate should be corrected in the state model; a disputed service priority should go to the policy owner; an unauthorized action should be blocked, irrespective of its apparent benefit.

6. Transfer I: The Federal Government as a Dynamic Allocation System

A federal government is an unusually rich transfer case because it contains many dependencies, programs, legal mandates, appropriations, procurement processes and implementation calendars. Programs do not begin or mature simultaneously. Some expenditures are entitlement-like; others are discretionary. Some projects are multi-year. Others depend on seasonal windows, matching funds, legislative authorization or administrative capacity.

At the beginning of a fiscal period, the government may possess an allocation and implementation sequence that is internally coherent. The analogy with the delivery network is direct: programs are commitments; ministries and agencies are operating nodes; budget authority and administrative capacity are scarce resources; deadlines and statutory mandates are service constraints.

Now introduce a contingency: a hurricane, pandemic, migration shock, banking crisis, drought, cyberattack or abrupt revenue shortfall. The question is not merely how to “optimize spending.” The institution must decide whether previously valid priorities should be reordered in time.

FEDERAL-GOVERNMENT TRANSFER

A self-driving public-allocation system would not give an LLM authority to move appropriations. Deterministic systems would first identify legal availability, fiscal constraints, procurement capacity and feasible reallocations. A bounded judgment layer could then compare admissible sequences: accelerate emergency health procurement, delay lower-priority capital expenditure, protect transfers with legal priority, preserve critical infrastructure, or escalate decisions that require political or legislative authority.

A useful state vector might include cash availability, budget authority, execution rates, legal earmarks, procurement lead times, regional stress indicators and program-level social objectives. The candidate generator would produce only legally feasible timing adjustments. Judgment would operate on questions such as urgency, irreversibility, distributional consequences and resilience. Governance would encode constitutional, statutory and appropriations constraints. Auditability is indispensable because public authority cannot be reduced to model confidence.

This framework also clarifies why calendars matter. A program can have a positive net social value and still be optimally delayed after a contingency if another program has become temporarily dominant. Conversely, a program that appears inefficient in a narrow cost metric may become non-deferrable because of legal, humanitarian or systemic-risk considerations. The lesson is not that “AI should run government.” It is that dynamic public administration can be modeled as governed re-sequencing under changing priorities.

Public allocation as a constrained sequence

A practical public-sector analogue begins with the distinction between money that is legally available and money that is merely visible in a budget table. A system must represent appropriation purpose, time limits, matching requirements, obligated amounts, procurement rules and the office authorized to approve a change. It should also represent execution capacity: an agency cannot accelerate a program simply because funding exists if staff, suppliers or delivery sites are unavailable. Candidate generation should reject changes that cross those boundaries before any prioritization discussion begins. A language model should not infer that statutory authority exists from a program description or from past practice.

Consider an emergency that creates simultaneous demands on public health logistics, transport infrastructure and local assistance. The initial plan may be to complete several capital projects before a seasonal deadline while maintaining ordinary service programs. The shock may make a medical supply contract urgent, but procurement lead time and local distribution capacity could limit the value of accelerating it. The model can help compare timing alternatives only after officials establish which funds can be used, which programs are protected and which decisions require legislative or executive approval. A recommendation to delay a project should disclose both the immediate capacity released and the downstream cost of missing its window.

Public priorities also involve distribution. An aggregate benefit measure can hide that a re-sequencing concentrates delay on a region, a vulnerable population or a group with limited alternatives. The decision record should therefore identify who receives an earlier service, who waits longer and how the policy treats comparable cases. This does not imply that every distributional conflict has a single quantitative answer. It means the system should surface the trade-off for accountable officials rather than bury it inside an efficiency score. Where policy is unsettled, escalation is a feature of good design, not a failure to automate.

A responsible pilot would begin with a narrow administrative queue rather than autonomous budget movement. It could offer ranked timing options for human review, preserve the original allocation, show legal and operational exclusions and record whether officials accepted the advice. Evaluation would examine timeliness, consistency, distributional effects, appeal or correction rates, and administrative burden. It would also test whether the system treats similar cases similarly when irrelevant wording or departmental identity changes. Only after authority, data

quality and review processes are established should a public institution consider increasing the degree of automation. This approach makes the transfer useful while respecting that public legitimacy depends on law, explanation and contestability, not just performance.

7. Transfer II: Complex Corporations and Multi-Subsidiary Groups

A diversified corporation faces the same architecture at a different scale. Subsidiaries compete for capital, liquidity, management attention, technology resources, risk capacity and sometimes scarce operational inputs. Annual budgets and strategic plans impose a sequence: factory expansion in one quarter, refinancing in another, system migration later, dividend upstreaming on a fixed schedule, and acquisitions conditional on leverage targets.

A contingency can make the original sequence locally irrational. A subsidiary may face a cyber incident, regulatory capital demand, supply-chain disruption, litigation shock or sudden growth opportunity. Headquarters must determine whether to preserve the existing capital plan or reprioritize.

The delivery analogy is powerful because the corporation also has heterogeneous “packages.” A covenant-sensitive refinancing, a safety-critical plant repair, a discretionary marketing campaign and a high-return acquisition cannot be treated as interchangeable claims on cash. The institution needs a hierarchy of commitments.

CORPORATE AUTONOMY

Optimization can generate feasible capital-allocation packages under leverage, liquidity, tax and covenant constraints. Judgment can decide which strategic objective should dominate under the new environment. Governance can enforce board authorities, subsidiary protections, related-party rules and risk limits. Execution can remain entirely deterministic and auditable.

Potential applications include treasury cash concentration, intercompany funding, capex sequencing, inventory allocation, insurance purchasing, cyber-response resources, shared-service capacity and crisis management. The key transfer is not the delivery algorithm. It is the separation of *feasible set*, *priority judgment*, *authority*, and *state transition*.

Group-level coordination and entity boundaries

A corporate group is not a single undifferentiated wallet. Subsidiaries may be separate legal entities with their own creditors, regulators, minority shareholders, tax positions, licenses and local obligations. A group optimizer might identify an apparent benefit from moving liquidity or delaying investment, but that move can be unavailable or harmful once transfer restrictions, covenants, operational dependencies and local solvency are considered. The feasible set must therefore encode both group-wide limits and entity-specific protections. Escalation is necessary whenever the relevant authority belongs to a subsidiary board, lender or regulator rather than to headquarters alone.

A useful scenario is a sudden shortage of working capital at one operating company during a broader market shock. The initial capital plan may reserve cash for a growth project at another subsidiary and schedule a refinancing for a later date. Candidate alternatives could include drawing committed facilities, postponing discretionary capex, arranging permitted intercompany funding or accelerating a refinancing. Each option has different consequences for liquidity buffers, covenant headroom, cost and resilience. A bounded judgment layer can help articulate which objective becomes dominant, but deterministic models must calculate cash flows and constraints, and treasury or legal officers must confirm the assumptions and authority.

The same architecture applies to nonfinancial resources. During a cyber incident, scarce security engineers, forensic capacity and executive attention must be assigned across business units. A schedule optimized for routine ticket closure may be inappropriate if an incident threatens a critical service or regulated data. Yet declaring one event “critical” can create a queue-jumping incentive. The institution needs consistent severity criteria, evidence about dependencies, and a review mechanism for challenged classifications. The model may compare the documented cases, but should not be able to raise a unit’s priority merely because its narrative sounds more urgent.

Corporate use also requires care around incentives. A subsidiary that expects headquarters to rescue it after every shock may underinvest in resilience. If the system repeatedly redirects group resources toward the most visible short-term problem, it may penalize units that followed the approved plan and reward weak preparation. Evaluation should consequently measure not only immediate group returns but also the distribution of scarce resources, repeat emergency requests, local buffer adequacy and long-term strategic commitments. Managers need to know which decisions the system can recommend, which require committee approval and how exceptions are documented.

A sensible implementation would begin as a decision-support process for a recurring, bounded allocation, such as sequencing noncritical capex under a fixed liquidity envelope. The pilot would preserve board-approved floors, debt covenants, tax constraints and critical maintenance. Human committees would review any recommendation that changes legal-entity exposures or breaches strategic thresholds. If the process improves speed and consistency without weakening local controls, its scope can expand gradually. The relevant measure of success is not maximum centralization; it is the ability to coordinate a group response while preserving the rights and constraints that make the group a federation of accountable entities. Before any automated support is introduced, leadership should specify the allocation objective in terms that subsidiaries can contest and understand. If the system values group liquidity

above unit growth, that hierarchy should be approved through the ordinary governance process and tested against stress conditions. Reporting should show not only the proposed transfers but also which alternatives were excluded and why. The same explanation should be available to the unit whose proposal was deferred, so the system does not become an opaque central veto.

8. Transfer III: Finance—Where Re-Sequencing Is Already Everywhere

Finance is especially suitable because many systems are already autonomous at the optimization layer. The opportunity is to add bounded judgment without surrendering hard controls.

Portfolio management and trading

A portfolio optimizer may target return, tracking error, turnover, liquidity and factor exposure. Under ordinary conditions the weights are stable. During a volatility shock, liquidity event or regime change, the ordering can shift: preserving liquidity may dominate expected alpha; reducing concentration may dominate transaction cost; avoiding forced sales may dominate tracking error. A judgment layer can select among optimizer-generated portfolios, while pre-trade controls retain authority.

Bank balance-sheet and liquidity management

Banks continuously allocate scarce balance-sheet capacity across lending, securities, market-making and liquidity buffers. Stress can change the priority ordering. Liquidity coverage, collateral availability and deposit stability may become dominant over margin. The analogy to an urgent medical package is not rhetorical: some commitments become institutionally protected because failure has nonlinear consequences. Basel liquidity and risk-data principles illustrate why measurement, limits and governance must remain explicit [3; 4].

Collateral and margin management

Collateral optimization normally minimizes funding cost while satisfying eligibility and haircut constraints. A market shock can change what matters: optionality, settlement certainty, intraday liquidity or preservation of high-quality liquid assets may dominate nominal carry. A bounded judgment layer could choose among feasible collateral schedules produced by deterministic optimization.

Credit and restructuring workflows

Credit committees often face sequential decisions: renew, amend, reduce, collateralize, escalate or exit. New information can reorder priorities. A deterioration in borrower liquidity, a legal event or a systemic-sector shock may justify moving a case ahead of others even when the original review calendar was rational. Here judgment is naturally institutional and should be constrained by policy, delegated authorities and documentation requirements.

Insurance and claims

A catastrophe creates a surge in claims that cannot all be processed with equal speed. Deterministic systems can establish coverage, severity estimates and feasible workflows. Judgment can prioritize life-safety, vulnerable policyholders, business interruption or fraud review under explicit policy. Governance remains essential because prioritization can create conduct and fairness risks.

Treasury, settlement and payment operations

Corporate and bank treasuries manage queues of payments, funding needs and settlement obligations. Under normal conditions the sequence can minimize cost. Under stress, preventing settlement failure, preserving daylight liquidity or protecting a critical counterparty may dominate. This is almost a direct translation of the delivery experiment: the packages become obligations, vehicles become liquidity channels, routes become funding paths, and deadlines become settlement cutoffs.

FINANCIAL TRANSFER PRINCIPLE

The strongest applications are not those in which an LLM replaces an optimizer. They are those in which a mature optimizer already exists, the feasible set is well defined, and the missing capability is contextual re-prioritization among objectives under a changed state.

Finance-specific implementation conditions

The finance examples share a common pattern but should not be treated as interchangeable. Portfolio rebalancing is typically evaluated through risk exposures, liquidity, mandate compliance and market impact. Collateral allocation depends on eligible assets, haircuts, location and settlement timing. Credit review involves borrower information, legal rights and consistency across comparable obligors. Claims handling carries policyholder duties, fraud controls and conduct considerations. Payment sequencing is constrained by cutoffs, available funds, payment-system rules

and the consequences of a missed settlement. The candidate generator and governance rules must be tailored to each workflow rather than copied from the delivery simulator.

These distinctions suggest a prudent integration design. The judgment service should receive a versioned snapshot of the relevant positions or cases, not broad access to trading books or customer records. It should return a recommendation over precomputed choices, with a reason code tied to policy considerations. Existing limit engines, approval workflows and execution systems should remain authoritative. A recommendation that would increase risk, breach a mandate or delay a protected obligation must be rejected by deterministic controls regardless of the model's explanation. The model should not be able to alter the data on which the independent control relies.

A financial institution should also compare the costs of intervention with the expected benefit. Model calls create latency, operational dependencies, privacy considerations and new validation needs. For routine cases with a stable objective and clear ranking, a deterministic rule may be cheaper, faster and more consistent. The LLM layer is most defensible when the case presents a material, documented conflict among objectives and when the model can add contextual comparison without substituting for expert review. Monitoring should report intervention and override rates, not just average portfolio return or processing speed.

There are important limits to the analogy. A parcel's deadline and a bank's liquidity obligation may both be time-sensitive, but failure consequences, legal treatment and externalities differ substantially. Financial markets are strategic environments in which counterparties react; the synthetic delivery network is not. A policy that works under fixed simulated shocks may fail when actors adapt to it or when the data reflects past stress patterns poorly. Any transfer therefore requires domain-specific validation, stress testing and approval from risk, compliance and business owners. The monograph's contribution is to identify where re-prioritization can be studied, not to establish that one generic model is fit for every financial decision.

9. A Taxonomy of Transfer Learning from the Delivery Network

The phrase *transfer learning* is used here in an architectural and institutional sense, not only in the machine-learning sense of reusing model weights across datasets. What transfers is the decomposition of the decision problem.

Delivery concept	Government analogue	Corporate/financial analogue	What transfers
Package	Program or public commitment	Payment, loan, claim, trade, project	Unit of obligation
Vehicle capacity	Administrative/fiscal capacity	Capital, liquidity, risk capacity	Scarce execution resource
Deadline	Statutory/program milestone	Settlement, covenant, maturity, SLA	Time-dependent constraint
Service class	Legal/social priority	Seniority, criticality, client tier	Heterogeneous institutional value
Traffic/weather shock	Emergency/revenue shock	Market, cyber, liquidity shock	Exogenous state change
Candidate set	Legally feasible reallocations	Limit-compliant alternatives	Bounded action space
Claude judgment	Policy prioritization support	Supervisory/committee judgment	Contextual objective ordering
Governance gate	Legal/appropriation authority	Limits, mandates, delegated authority	Permission to act
Audit trail	Administrative record	Model/risk/compliance record	Accountability
Feedback	Program outcomes	P&L, risk, liquidity, service outcomes	Closed-loop adaptation

Five additional transfer ideas follow.

1. **Attention allocation:** executive attention itself can be modeled as a scarce resource whose sequence changes under shocks.
2. **Regulatory queues:** licensing, examinations, enforcement and approvals can be re-sequenced when systemic or consumer-protection risk changes.
3. **Central-bank operations:** liquidity facilities, collateral operations and supervisory attention can be reprioritized under market stress, while legal mandates remain hard constraints.
4. **Infrastructure portfolios:** maintenance schedules can be reordered when failure probabilities, weather or public-safety consequences change.
5. **Healthcare capacity:** beds, staff, diagnostics and elective procedures form a governed queue in which urgency can legitimately override the baseline schedule.

The transfer test is simple: does the target domain contain heterogeneous commitments, scarce resources, deadlines, environmental shocks, multiple legitimate objectives and explicit authority constraints? If so, the delivery architecture is likely to be informative.

What transfers, and what must be rebuilt

Transfer can be divided into four layers. The first is representational: identify the institution's commitments, resources, deadlines, service classes and state changes. The second is computational: estimate consequences, construct feasible alternatives and retain a deterministic benchmark. The third is normative: specify which objectives can be traded off, which have protected status and who may change their priority. The fourth is operational: connect recommendations to approval, execution, logging and feedback. The first two layers often admit technical reuse; the third and fourth are specific to the receiving institution and cannot be imported from a parcel example by changing labels.

A practical transfer exercise should begin with a mapping workshop involving process owners, operators, risk specialists and people affected by the queue. For each delivery concept, the group should identify a domain analogue

and test whether the analogy preserves the important properties. “Vehicle capacity” might map to analyst hours in one workflow and intraday liquidity in another, but the relevant constraints differ. “Urgent package” might mean a statutory deadline, a patient-safety concern or a settlement cutoff. Those cases should not be assigned the same service hierarchy simply because they share a time-pressure structure.

The next step is to document non-transferable assumptions. The synthetic city assumes a controller can observe a disruption and a simulator can update the state. Government data may be delayed across agencies; corporate subsidiaries may report on different calendars; financial decisions may change market prices and counterpart behavior. The target institution must decide how to manage missing data, stale signals, strategic responses and contested classifications. It must also determine whether the system is advising a person, prioritizing a queue or initiating an operational change. Those are materially different uses with different review obligations.

Transfer should be treated as a sequence of maturity gates rather than as a one-time model port. A desk exercise can test whether the vocabulary is useful. A retrospective replay can compare proposed priorities with historical decisions. A shadow deployment can produce advice that does not affect operations. A controlled pilot can then measure the effects within a narrow delegated scope. At every stage, owners should define a stop condition: unacceptable constraint violations, unexplained disparate impacts, excessive overrides, unreliable state reconstruction or weak evidence of benefit. This prevents a successful demonstration from becoming a deployment argument by momentum alone.

Finally, transfer requires preserving institutional pluralism. Different organizations may reasonably rank resilience, speed, fairness, return and continuity differently. The model should not silently standardize those choices. Its value is to make the consequences of alternative orderings more explicit and to help operators compare them against approved policy. A system is not more transferable because it applies the same weights everywhere. It is more transferable when its architecture accommodates different legitimate objectives while keeping the process of setting those objectives visible and accountable.

10. Governance: Why Judgment Must Not Become Unbounded Discretion

The most important risk in “optimization with judgment” is that the judgment layer becomes an opaque mechanism for overriding policy. The notebook avoids this by bounding both inputs and outputs. The LLM sees a compact evidence packet; it chooses from a finite set; its identifier is validated; the governance gate retains execution authority; and provenance is recorded.

For consequential domains, this should be strengthened further. The NIST AI Risk Management Framework emphasizes governance, measurement and lifecycle risk management [8]. Financial model-risk guidance similarly stresses independent validation, effective challenge and controls around model use [12]. These ideas map naturally onto the notebook.

NON-NEGOTIABLE CONTROL PRINCIPLES

- The LLM must not define its own authority.
- Hard legal, safety, capital, liquidity and mandate constraints remain deterministic.
- Every recommendation must identify the feasible alternative selected and the evidence used.
- Material ambiguity should increase escalation, not improvisation.
- Ex-post evaluation must compare judgment against a baseline and distinguish difference from improvement.

The architecture is therefore compatible with a “human-on-the-loop” model: routine exceptions can be handled autonomously inside delegated bounds, while material or ambiguous cases return to human authority. The purpose of automation is not to eliminate governance but to make governance executable.

A control stack for bounded judgment

Governance should be designed as a control stack that spans the full lifecycle. Before use, the institution should define the decision purpose, permitted users, affected parties, action boundaries and accountable owner. Data controls should establish provenance, freshness, access rights and handling requirements. The candidate service should expose constraints and exclusions in a form that can be independently checked. The language model should receive only the information needed for the bounded choice, and its output should be treated as untrusted until validated. After validation, a separate execution service should apply the action and record the result.

The control stack also needs explicit authority tiers. Low-impact, reversible timing changes may be delegated within narrow limits. Actions that alter a protected service, affect a large exposure, create an irreversible commitment or depend on contested evidence should require human approval. The threshold should be defined in policy before a case arises, not improvised because a model expressed confidence. Escalation routes must name a responsible role, expected response time and safe interim behavior. If no authorized reviewer is available, the system should follow a conservative default rather than expanding its own discretion.

Independent challenge should test both the model and the rules around it. Reviewers can replay cases with counterfactual details, check whether equivalent cases receive equivalent treatment and probe whether irrelevant phrasing changes the ranking. They should also test adversarial or malformed inputs, stale data, missing candidates and conflicting policy instructions. A recommendation can be internally consistent and still be unacceptable if the candidate generator encoded a mistaken constraint or if the underlying priority policy lacks authorization. Effective challenge must therefore reach beyond prompt wording to the complete decision pipeline.

Monitoring should distinguish several risk classes. Input risk concerns errors, omission and data staleness. Model risk concerns unstable or unsupported recommendations. Process risk concerns missed approvals, execution mismatch or broken logging. Outcome risk concerns service failures, financial losses or distributional harm. Governance risk concerns unclear authority, unresolved overrides or policy drift. Indicators should be linked to actions: pause a model version, narrow its scope, require extra review, or return a workflow to the deterministic baseline. A monitoring dashboard that only reports average accuracy is not an adequate control system for decisions involving multiple objectives.

The institution should maintain versioned records of the model, prompt, policies, candidate-generation code and thresholds used in each decision. This makes it possible to investigate whether a changed result came from new evidence, a new model version or a revised institutional rule. Retention and privacy requirements should be addressed at design time, especially if prompts contain sensitive financial, employee or citizen information. Records should be detailed enough for accountability without becoming an uncontrolled duplicate of confidential source systems. Governance is thus not a final approval box; it is the set of institutional arrangements that make each earlier

layer answerable to a defined authority. Human review should be meaningful rather than ceremonial. Reviewers need enough time, authority and evidence to reject a recommendation, and the system should record whether they accepted, modified or overrode it. If operators routinely accept every suggestion because of workload or automation bias, a nominal approval gate does not provide effective control. If they routinely override the system, that pattern may indicate a faulty priority rule, poor input quality or a mismatch between formal policy and operational reality. Both acceptance and disagreement therefore deserve monitoring.

Governance must also address change management. A model update, revised prompt, altered candidate generator or new policy threshold can change decisions even when the visible workflow appears unchanged. Changes should be versioned, tested on a representative scenario set and approved by the accountable owner before entering production. Material incidents should trigger a documented review and, where appropriate, temporary reversion to the deterministic baseline. A system that cannot be paused safely is not meaningfully governed.

11. What Cell 16 Adds: From Demonstration to Research Design

Cell 16 is more than an extra dashboard. It introduces the beginnings of an empirical research design. For each actual Claude decision, the system reconstructs the deterministic choice and asks whether judgment altered the action. It then measures the characteristics of the divergence.

This can be extended into a stronger experimental program. First, run repeated synthetic days under fixed seeds and different shock intensities. Second, compare multiple judgment policies: deterministic baseline, Claude, human experts and perhaps alternative LLMs. Third, score not only immediate service and cost but downstream network fragility. Fourth, separate calibration from evaluation so that policy weights are not tuned on the same scenarios used to claim improvement. Fifth, classify disagreements by reason: protected-service priority, deadline rescue, robustness, cost, or escalation.

A particularly useful metric is *selective divergence*: the proportion of cases in which judgment differs from the optimizer, conditional on the case actually containing a material trade-off. Good judgment should not disagree for the sake of disagreement. It should preserve routine optimization when the state is ordinary and intervene when the objective hierarchy has genuinely changed.

THE RESEARCH HYPOTHESIS

The value of judgment is not a higher raw disagreement rate. It is a better mapping from context to priority: low intervention in routine states, selective re-sequencing in exceptional states, and escalation when authority or evidence is insufficient.

A stronger evaluation protocol

A research program should pre-register the principal questions and outcomes before inspecting comparative results. One question is whether judgment changes action at all. A second is whether changes are concentrated in scenarios with a predefined material trade-off. A third is whether the changed actions improve the institution's stated outcomes without violating constraints or shifting excessive harm to other groups. These questions should be kept separate. High disagreement may indicate useful responsiveness, inconsistent behavior or simple noise; only the accompanying context and outcomes distinguish those explanations.

The basic experimental unit should be a complete scenario, not an isolated model call. Scenarios can vary the type, timing and severity of shocks while preserving common random seeds across policies. Each policy should receive equivalent information and the same feasible action set, and each should start from the same initial state. The deterministic baseline, bounded LLM policy and human decision-makers can then be compared under matched conditions. Repeated runs are needed because one unusual event sequence can dominate an average. Results should report distributions and uncertainty, not only a single favorable example or a mean score.

Outcome measures should include constraint violations, service completion, lateness by class, direct cost, recovery time, resource utilization and fragility under subsequent shocks. If the application has distributional effects, outcomes should be disaggregated by relevant group or region. Process measures should include model availability, latency, invalid response rate, escalation frequency, fallback frequency and human override rate. A policy that modestly improves service but requires extensive correction may not be operationally valuable. Conversely, a policy that does not change aggregate cost may still be useful if it reduces severe tail failures, provided the risk preference was specified in advance.

Human comparison requires care. Experts should receive the same case packet as the model and make decisions independently before discussion. Their rationales can be coded against the same policy categories, but inter-rater disagreement should be reported rather than collapsed into a fictitious single ground truth. A panel can later adjudicate cases, while preserving original opinions for analysis. If model prompts or policy weights are tuned after observing errors, that tuning data should be separated from the final test set. Otherwise, apparent improvement may simply reflect repeated adaptation to the evaluation scenarios.

Finally, replay studies do not establish real-world impact. They can test consistency and expose failure cases, but a shadow deployment is needed to learn about data latency, operator trust and workflow burden. A later controlled pilot should have explicit scope, rollback criteria and independent monitoring. The Cell 16 local counterfactual is an instructive starting point because it compares the choices at one state; full policy rollouts are the stronger next step because actions alter future states. The evaluation claim must match the method: a decision-point comparison supports a statement about divergence at that point, not a general claim that the LLM improves institutional performance. Research reports should include null and unfavorable results. If judgment rarely changes decisions, that may mean the deterministic baseline already captures the relevant priorities, or it may mean the model lacks useful context. If it changes choices but worsens outcomes, the analysis should identify whether the error arose from poor evidence, an unsuitable hierarchy or a weak candidate set. Publishing only selected successful cases encourages

confirmation bias and makes it difficult for other researchers to reproduce the finding.

The research record should preserve enough detail for an independent analyst to reconstruct each comparison while protecting sensitive information. At minimum, this includes scenario definitions, random seeds, policy versions, outcome definitions, exclusions and missing-data treatment. Evaluation should also distinguish exploratory analysis from confirmatory tests. Such discipline is essential because the central claim is conditional: judgment may help in some changing-priority cases, but that benefit must be demonstrated rather than inferred from model fluency or disagreement with a baseline.

12. Conclusion

The self-driving delivery network begins as a logistics example and ends as a theory of institutional autonomy. Its central lesson is that an optimized sequence is conditional on a state of the world and a hierarchy of objectives. When either changes, blindly preserving the sequence can become the opposite of intelligent behavior.

The notebook therefore separates optimization from judgment. Deterministic methods represent the world, calculate travel times and risk, generate feasible alternatives, enforce constraints and execute authorized actions. Claude Haiku 4.5 enters at a narrower but important margin: it chooses among feasible alternatives when the institution must decide which objective deserves priority. Governance remains independent of the model, and feedback makes the system closed-loop rather than episodic.

This architecture transfers naturally to finance because financial institutions already contain sophisticated optimization engines. Portfolio construction, collateral allocation, liquidity management, treasury, settlement, credit and insurance all involve scarce resources, deadlines and multiple objectives. Their hardest moments are often not moments in which optimization disappears, but moments in which the priority ordering changes. Liquidity dominates return; settlement certainty dominates carry; systemic risk dominates local profitability; resilience dominates efficiency.

The same logic applies to government. A federal administration is a portfolio of programs with different calendars, legal authorities, social objectives and implementation capacities. A contingency can make the original temporal allocation inappropriate. The correct autonomous architecture would not allow a language model to move public money. It would use deterministic systems to identify legal and operational alternatives, bounded judgment to reason about changed priorities, governance to preserve constitutional and delegated authority, and audit trails to explain the resulting sequence.

Complex corporations provide a third domain. Capital budgets, subsidiary liquidity, cyber response, capex, refinancing and strategic initiatives are all sequenced under assumptions. When assumptions change, headquarters must decide whether the sequence should change too. Again, the problem is not simply optimization. It is optimization with judgment.

The broader proposition is therefore precise. Autonomous institutions need at least four distinct capabilities: **measurement**, **optimization**, **judgment**, and **governance**. Measurement tells the institution what appears to be happening. Optimization constructs efficient feasible responses. Judgment determines which objective deserves priority in context. Governance determines what the institution is allowed to do. Execution and feedback then convert these layers into adaptive behavior.

The delivery network makes this architecture easy to see because the consequences are intuitive: a medical package can move ahead of an ordinary parcel; a failed vehicle can force reassignment; weather can make yesterday's route wrong. The same phenomenon is less visually obvious but more consequential in finance and government. A mature autonomous system should therefore be judged not by whether it can preserve a plan, but by whether it can recognize when the plan's ordering has ceased to serve the institution's objectives—and change that ordering without abandoning control.

The computational implementation accompanying this monograph is the Colab notebook `Self_Driving_Delivery_Network_LLM`. The notebook should be read together with the monograph: the text develops the institutional argument, while the notebook makes the architecture executable, observable and testable.

FINAL TAKEAWAY

The next generation of autonomous systems will not be defined by optimization alone and not by LLMs alone. Their defining capability will be governed re-prioritization: preserving hard constraints while changing the sequence of action when the environment changes what matters most.

Implications for institutional design

The practical implication is not that every organization should insert a language model into its control loop. It is that institutions should make visible the points at which operating assumptions, objective rankings and authority are currently bundled together. A team may discover that a routine scheduling rule already encodes an implicit priority. Making that rule explicit can improve accountability even if no LLM is used. In other cases, a bounded language model may help compare exceptions that are difficult to reduce to a stable score, while deterministic systems retain control over facts, constraints and execution.

The transition should therefore be incremental. Start with a well-defined workflow, document the current decision rights and preserve a transparent baseline. Test whether the system identifies the intended exceptions, whether humans can understand its recommendations and whether independent controls catch invalid choices. Only then consider broader delegation. The institution should remain able to revert to the prior process, explain why an

override occurred and identify who accepted responsibility. Autonomy is valuable when it improves responsiveness without making the path from evidence to action less intelligible.

The delivery example also marks a limit. Parcels and vehicles are easier to simulate than rights, markets, public legitimacy or organizational politics. A technically accurate state estimate cannot decide whose interests count, and a useful priority recommendation cannot create legal authority. Transfer therefore depends on domain expertise and participation by those affected, not merely on software adaptation. The strongest claim supported by this monograph is architectural: adaptive institutions benefit from separating measurement, optimization, judgment, governance and feedback so each can be tested and assigned to the appropriate actor.

References

Reading guide

The sources below connect the monograph's operational, organizational and governance claims. They are not presented as evidence that a language model is superior to a conventional optimizer. Instead, they provide foundations for understanding feedback, constrained decision-making, vehicle routing, organizational adaptation and the control of models in consequential settings. The reader can follow the list by question: how does a system revise action as state changes; how do organizations make decisions under bounded information; what makes a routing problem computationally tractable; and what safeguards are needed when models influence institutional processes?

The feedback and control tradition is represented by Åström and Murray, Bellman, Bertsekas, and Rawlings, Mayne and Diehl. Together, these works help distinguish state, policy, prediction and repeated action. They are relevant to the notebook because its decision is not a one-time route calculation: an action changes the state from which later choices are made. The companion's simplified simulator does not implement the full machinery of model predictive control or dynamic programming, but those frameworks clarify why replanning must use updated state and why a local decision can affect future options.

Simon and March with Simon provide an organizational lens on bounded rationality, rules and adaptation. Weick's work on sensemaking is useful for considering how institutions interpret ambiguous events before acting. These perspectives complement the quantitative model: organizations do not merely calculate over a fixed objective; they notice, classify and respond through routines and authority structures. Rahwan and colleagues' discussion of machine behavior further situates algorithmic systems as objects of empirical study, encouraging researchers to examine what systems do in practice rather than infer institutional value from design intent alone.

Toth and Vigo provide the domain-specific routing reference behind the synthetic delivery setting. The monograph uses a deliberately modest route model and should not be read as implementing or benchmarking the full range of vehicle-routing methods surveyed in that field. Amershi and colleagues' human-AI interaction guidelines are relevant to the design of understandable recommendations and appropriate user expectations, particularly where an operator needs to distinguish advice from an authorized action.

The governance references address different layers of risk. NIST's AI Risk Management Framework offers a lifecycle-oriented vocabulary for governing, mapping, measuring and managing AI risks. Federal Reserve and OCC model-risk guidance supplies a financial supervisory frame for independent validation, effective challenge and controlled use. Basel's liquidity principles and BCBS 239 address the importance of liquidity-risk monitoring and effective risk-data aggregation in banking. These sources are not interchangeable, and the monograph does not claim that any one document directly prescribes an LLM architecture. They support the narrower argument that model-supported decisions in institutions require reliable data, clear ownership, limits, validation and reporting.

Taken together, the bibliography is a map for extending the exercise rather than a closed canon. Operations researchers can strengthen the candidate generator and dynamic policy evaluation. Organizational scholars can investigate how formal priorities interact with routines and discretion. Public-administration and financial researchers can specify domain-specific authority and outcome measures. Human-factors and governance specialists can test whether explanations, escalation paths and audit records support responsible use. Future empirical work should add domain sources and current policy materials for each deployment context, rather than treating this synthetic logistics study as a substitute for sector-specific research.

How to extend the reading

Readers should distinguish foundational theory, domain guidance and empirical evidence when using this bibliography. The foundational works provide concepts for reasoning about feedback, organizational choice and routing, but they do not test this particular notebook or establish that an LLM improves outcomes. The supervisory and governance materials describe control concerns in their respective institutional settings; they should be consulted in their full context before being applied to a new workflow. Human-AI interaction guidance can inform interface design, but it cannot replace legal, operational or model validation. Keeping these evidence types separate avoids turning a relevant citation into a claim stronger than the cited work supports.

A next research step is to build a domain-specific literature review around the target application. A public program would need current statutory and administrative sources, evaluation research on the relevant service, and evidence about who bears delay. A banking application would need current prudential requirements, institution-specific risk policy and empirical work on the market or balance-sheet process. A corporate deployment would need governance documents, entity constraints and operational evidence from the affected subsidiaries. The delivery references offer a vocabulary for constructing these investigations, not a shortcut around them.

The bibliography is intentionally interdisciplinary because the architecture crosses technical and institutional boundaries. Control theory helps describe how action and state evolve; operations research clarifies how to create

feasible alternatives; organization theory explains why routines and bounded attention matter; governance frameworks highlight responsibility and lifecycle controls; and human-factors work asks whether people can use the system as intended. A complete deployment study would add stakeholder research, privacy and security analysis, impact assessment, procurement review and domain-specific validation. These additions should be selected according to the system's actual role and affected population rather than appended as generic formalities.

The sources should also be read critically. A framework is not proof that a specific implementation complies with it, and a supervisory document for one kind of model use does not automatically determine the treatment of every generative system. A theory can motivate a design distinction without proving its practical benefit. An empirical finding from one population or operating environment may not generalize to another. The monograph therefore uses references to orient the reader and support bounded conceptual claims, while leaving sector-specific conclusions to research that tests the relevant system, policy and institutional context directly.

References

- [1] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2010.
- [2] S. Amershi et al., “Guidelines for Human-AI Interaction,” *Proceedings of CHI*, 2019.
- [3] Basel Committee on Banking Supervision, *Basel III: The Liquidity Coverage Ratio and Liquidity Risk Monitoring Tools*. Bank for International Settlements, 2013.
- [4] Basel Committee on Banking Supervision, *Principles for Effective Risk Data Aggregation and Risk Reporting (BCBS 239)*. Bank for International Settlements, 2013.
- [5] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.
- [6] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, 4th ed. Athena Scientific, 2017.
- [7] J. G. March and H. A. Simon, *Organizations*. Wiley, 1958.
- [8] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, 2023.
- [9] I. Rahwan et al., “Machine Behaviour,” *Nature*, vol. 568, pp. 477–486, 2019.
- [10] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed., 2017.
- [11] H. A. Simon, *Administrative Behavior*. Macmillan, 1947.
- [12] Board of Governors of the Federal Reserve System and Office of the Comptroller of the Currency, *Supervisory Guidance on Model Risk Management (SR 11-7 / OCC 2011-12)*, 2011.
- [13] P. Toth and D. Vigo, eds., *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. SIAM, 2014.
- [14] K. E. Weick, *Sensemaking in Organizations*. Sage, 1995.